

Cryptography Overview

WPC

2025 年 9 月 14 日

目录

1	古典密码学	3
1.1	代换密码	3
1.1.1	Caesar 密码	3
1.1.2	Vigenère 密码	3
1.1.3	Playfair 密码	3
1.1.4	Hill 密码	4
1.2	置换密码	4
2	流密码	5
2.1	线性反馈移位寄存器	5
2.2	非线性组合子系统	5
2.3	RC4	5
3	分组密码	7
3.1	DES	7
3.1.1	DES 简介	7
3.1.2	初始置换	8
3.1.3	轮函数	8
3.1.4	密钥扩展	10
3.1.5	TDES	11
3.2	AES	11
3.2.1	AES 简介	11
3.2.2	轮函数	12
3.2.3	密钥扩展	14
3.3	IDEA	14
3.3.1	IDEA 简介	14
3.3.2	轮函数	14

3.3.3	输出变换	15
3.3.4	子密钥产生	15
3.4	SM4	15
3.4.1	SM4 简介	15
3.4.2	轮函数	15
3.4.3	密钥扩展	16
4	哈希函数	17
4.1	哈希函数简介	17
4.2	MD5	17
4.3	SHA-1	18
4.4	MD6	18
4.5	SHA-2、SHA-3 及 SM3 简介	18

1 古典密码学

1.1 代换密码

代换密码 (Substitution Cipher) 是用不同的字母替代明文中的字母, 从而达到加密目的。

1.1.1 Caesar 密码

Caesar 密码是一种单表代换密码 (Monoalphabetic Cipher), 它大约出现于公元前 100 年的高卢战争期间, 当时的古罗马统治者尤利乌斯·凯撒 (Julius Caesar) 使用其在军事通信中保护机密信息。在这种密码中, 每个字母都被替换为其后第 k 个字母。例如, $k = 3$ 时, A 替换为 D, B 替换为 E, 以此类推。N 就是密钥, 加密过程用公式表示:

$$c = (m + k) \bmod 26$$

1.1.2 Vigenère 密码

Vigenère 密码是由法国外交官布莱兹·德·维热内尔 (Blaise de Vigenère) 在 1586 年提出的一种多表代换密码 (Polyalphabetic Cipher)。它可以看作是 Caesar 密码的推广, 密钥和明文的长度一样长, 不足则重复密钥, 密钥中每个字符都对应一个 Caesar 密码的代换。加密过程用公式表示:

$$c_i = (m_i + k_i) \bmod 26$$

其中, m_i 是明文的第 i 个字符, k_i 是密钥的第 i 个字符, c_i 是密文的第 i 个字符。

1.1.3 Playfair 密码

Playfair 密码是由查尔斯·惠斯通 (Charles Wheatstone) 于 1854 年发明的一种分组代换密码 (Polygraphic Substitution Cipher), 因其好友莱昂·普莱费尔 (Lord Playfair) 的推广而以他命名。加密前先对明文分组, 分组过程:

1. 将明文分为两两一组。
2. 如果分组中两字符相同, 则插入 X 与第一个字符成组。
3. 如果字符数为奇数, 则在最后一个字符后面补上 X。

例如 “balloon” 被分组为 “BA LX LO ON”。之后是产生加密矩阵:

1. 选择一个密钥字符串, 删除其中重复字符。
2. 按顺序填入一个 5×5 的矩阵。
3. 矩阵之后的内容, 由字母表按顺序填充。

注意：在矩阵中，I 和 J 合并为一个字母，即用 I 替代 J，例如密钥为 “just “时的矩阵为：

<i>I</i>	<i>U</i>	<i>S</i>	<i>T</i>	<i>A</i>
<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>F</i>
<i>G</i>	<i>H</i>	<i>K</i>	<i>L</i>	<i>M</i>
<i>N</i>	<i>O</i>	<i>P</i>	<i>Q</i>	<i>R</i>
<i>V</i>	<i>W</i>	<i>X</i>	<i>Y</i>	<i>Z</i>

最后是加密过程：

1. 如果两字符在同一行，则将每个字符替换为其右边的字符（循环处理）。
2. 如果两字符在同一列，则将每个字符替换为其下方的字符（循环处理）。
3. 如果两字符在不同的行和列，则将它们替换为同一行中与它同组的字符所在列的字符。

1.1.4 Hill 密码

Hill 密码是由 Lester S. Hill 在 1929 年提出的一种多字母分组代换密码。其密钥为一个 $n \times n$ 的矩阵，且必须是可逆矩阵，明文被分为 n 个字符一组，一组为一个向量。加密过程用公式表示：

$$C = K \cdot M \bmod 26$$

1.2 置换密码

置换密码（Permutation Cipher）保持加明文加中的而所有字母，只是改变他们在明文中的位置，也被称作换位密码（Transposition Cipher）。置换密码的密钥为一个 n 元置换，加密时，先把明文分为 n 个字符一组，然后对每组字符进行置换。单轮置换容易被攻破，因此通常会进行多轮置换。

2 流密码

流密码 (Stream Cipher) 通常用于实时通信和数据流加密, 它将明文的每个字符与一个伪随机生成的密钥字符进行异或操作, 从而生成密文。按照密钥流是否独立于明文产生, 流密码分为同步流密码 (Synchronous Stream Cipher) 和自同步流密码 (Self-synchronizing Stream Cipher)。流密码的加密是直接把明文与密钥流进行异或操作, 公式为:

$$c_i = m_i \oplus k_i$$

其中, m_i 是明文的第 i 个字符, k_i 是密钥流的第 i 个字符, c_i 是密文的第 i 个字符。

2.1 线性反馈移位寄存器

线性反馈移位寄存器 (Linear Feedback Shift Register, LFSR) 是一种常用的伪随机数生成器, 流密码中密钥流的产生通常通过多个 LFSR 的并联或串联实现。GF¹(2) 上的 n 级 LFSR 由 n 个位存储器和一个反馈函数 $f(a)$ 组成, 当移位时钟脉冲到来时, 每一级存储器 a_i 都把内容一传递到下一级 a_{i-1} , 而最高位 a_n 则从反馈函数 $f(a)$ 中获取新值, a_1 为输出值。 n 级 LFSR 的输出序列 a_i 满足如下线性递推关系:

$$a_i = c_1 a_{i-1} \oplus c_2 a_{i-2} \oplus \cdots \oplus c_n a_{i-n}, \quad c_j \in \text{GF}(2)$$

这种关系可以用一个一元多项式表示, 这个多项式也称为 LFSR 的特征多项式 (Characteristic Polynomial):

$$P(x) = x^n + c_1 x^{n-1} + c_2 x^{n-2} + \cdots + c_n$$

当且仅当特征多项式为本原多项式 (Primitive Polynomial) 时, LFSR 可以生成 m 序列²。

2.2 非线性组合子系统

为了使得密钥流尽可能复杂, 常使用非线性组合子系统 (Nonlinear Combination Generator), 它通过多个 LFSR 的输出进行非线性组合来生成密钥流。非线性组合子系统通常使用逻辑门 (如与门、或门、异或门等) 对多个 LFSR 的输出进行组合, 这样可以增加密钥流的复杂性和安全性, 使得攻击者更难以预测或重建密钥流。常用非线性组合子系统有: Geffe 序列生成器、J-K 触发器、Pless 生成器、钟控序列生成器等。

2.3 RC4

RC4 (Rivest Cipher 4) 是由密码学家 Ron Rivest 在 1987 年设计的流密码。RC4 加密过程分为两部分: 密钥调度算法 (Key Scheduling Algorithm, KSA) 和伪随机生成算法 (Pseudo-Random Generation Algorithm, PRGA)。KSA 阶段, 先初始化一个长度为 256 的数组 S 盒, 再使用密钥打乱 S 盒的顺序。PRGA 阶段, 使用 S 盒生成伪随机字节流, 并与明文进行异或操作得到密文。

¹伽罗瓦域 (Galois Field)

²序列周期达到最大值 $2^n - 1$

Algorithm 1 RC4 Key Scheduling Algorithm (KSA)

```

1: procedure KSA(key[0...keylen-1])
2:   Initialize  $S[0 \dots 255] \leftarrow [0, 1, 2, \dots, 255]$ 
3:    $j \leftarrow 0$ 
4:   for  $i \leftarrow 0$  to 255 do
5:      $j \leftarrow (j + S[i] + \text{key}[i \bmod \text{keylen}]) \bmod 256$ 
6:     swap  $S[i]$  and  $S[j]$ 
7:   end for
8:   return  $S$ 
9: end procedure

```

Algorithm 2 RC4 Pseudo-Random Generation Algorithm (PRGA)

```

1: procedure PRGA( $S, \text{len}$ )
2:    $i \leftarrow 0, j \leftarrow 0$ 
3:   for  $k \leftarrow 0$  to  $\text{len} - 1$  do
4:      $i \leftarrow (i + 1) \bmod 256$ 
5:      $j \leftarrow (j + S[i]) \bmod 256$ 
6:     swap  $S[i]$  and  $S[j]$ 
7:      $t \leftarrow (S[i] + S[j]) \bmod 256$ 
8:      $K[k] \leftarrow S[t]$  ▷ 生成第  $k$  个密钥字节
9:   end for
10:  return  $K$ 
11: end procedure

```

Algorithm 3 RC4 Encryption

```

1: procedure RC4_ENCRYPT(plaintext, key)
2:    $S \leftarrow \text{KSA}(\text{key})$ 
3:    $K \leftarrow \text{PRGA}(S, \text{len}(\text{plaintext}))$ 
4:   for  $i \leftarrow 0$  to  $\text{len}(\text{plaintext}) - 1$  do
5:      $\text{cipher}[i] \leftarrow \text{plaintext}[i] \oplus K[i]$ 
6:   end for
7:   return cipher
8: end procedure

```

3 分组密码

3.1 DES

3.1.1 DES 简介

数据加密标准 (Data Encryption Standard, DES) 是由美国国家标准局 (National Institute of Standards and Technology, NIST) 在 1977 年采纳作为“非密级”应用的分组密码标准。DES 使用 64 位的分组长度和密钥长度 (其中有 8 位奇偶校验位)，它的加密过程包括 16 轮的迭代，每轮使用一个 48 位的子密钥。解密过程与加密过程类似，只不过密钥的使用顺序是相反的，即从第 16 轮到第 1 轮。

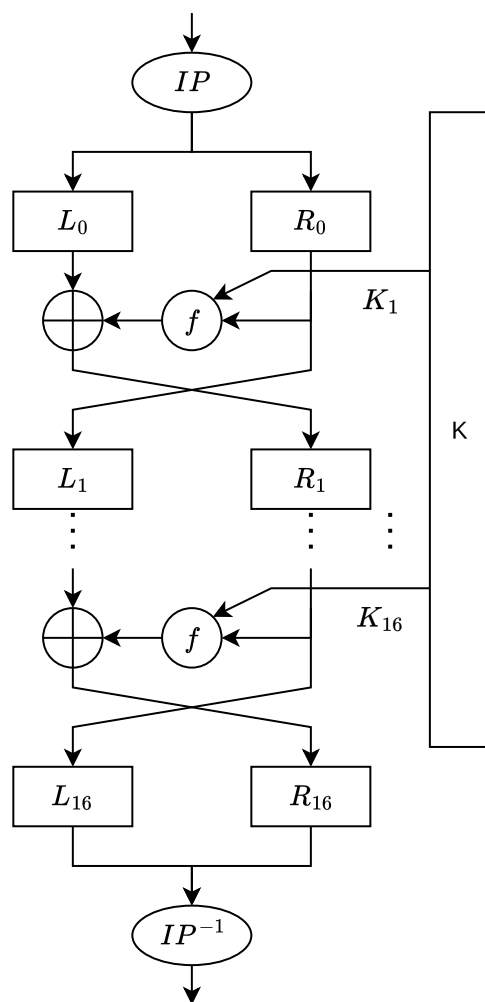


图 3.1: DES 加密流程

3.1.2 初始置换

初始置换 (Initial Permutation, IP) 是加密过程的第一步, 它对明文块 (64 位) 按一个固定的矩阵进行比特位置重排,

$$IP = \begin{bmatrix} 58 & 50 & 42 & 34 & 26 & 18 & 10 & 2 \\ 60 & 52 & 44 & 36 & 28 & 20 & 12 & 4 \\ 62 & 54 & 46 & 38 & 30 & 22 & 14 & 6 \\ 64 & 56 & 48 & 40 & 32 & 24 & 16 & 8 \\ 57 & 49 & 41 & 33 & 25 & 17 & 9 & 1 \\ 59 & 51 & 43 & 35 & 27 & 19 & 11 & 3 \\ 61 & 53 & 45 & 37 & 29 & 21 & 13 & 5 \\ 63 & 55 & 47 & 39 & 31 & 23 & 15 & 7 \end{bmatrix} \quad IP^{-1} = \begin{bmatrix} 40 & 8 & 48 & 16 & 56 & 24 & 64 & 32 \\ 39 & 7 & 47 & 15 & 55 & 23 & 63 & 31 \\ 38 & 6 & 46 & 14 & 54 & 22 & 62 & 30 \\ 37 & 5 & 45 & 13 & 53 & 21 & 61 & 29 \\ 36 & 4 & 44 & 12 & 52 & 20 & 60 & 28 \\ 35 & 3 & 43 & 11 & 51 & 19 & 59 & 27 \\ 34 & 2 & 42 & 10 & 50 & 18 & 58 & 26 \\ 33 & 1 & 41 & 9 & 49 & 17 & 57 & 25 \end{bmatrix}$$

3.1.3 轮函数

轮函数 (Round Function) 是 DES 加密的核心部分, 它将 32 位的输入数据与 48 位的子密钥进行处理, 输出 32 位的结果。轮函数的主要步骤包括:

1. 扩展置换 (Expansion Permutation), 将 32 位输入扩展为 48 位。
2. S 盒替换 (S-Box Substitution), 将 48 位数据分为 8 组, 每组 6 位, 每组输出 4 位。
3. P 盒置换 (P-Box Permutation), 32 位的置换。

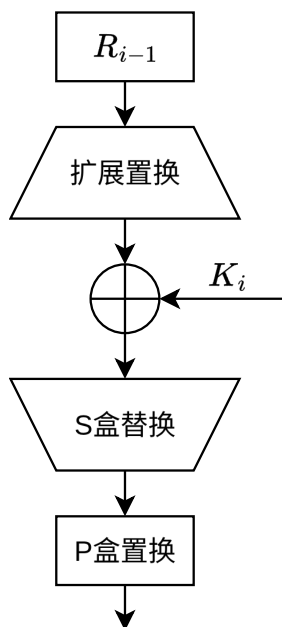


图 3.2: 轮函数结构

$$E = \begin{bmatrix} 32 & 1 & 2 & 3 & 4 & 5 \\ 4 & 5 & 6 & 7 & 8 & 9 \\ 8 & 9 & 10 & 11 & 12 & 13 \\ 12 & 13 & 14 & 15 & 16 & 17 \\ 16 & 17 & 18 & 19 & 20 & 21 \\ 20 & 21 & 22 & 23 & 24 & 25 \\ 24 & 25 & 26 & 27 & 28 & 29 \\ 28 & 29 & 30 & 31 & 32 & 1 \end{bmatrix} \quad P = \begin{bmatrix} 16 & 7 & 20 & 21 \\ 29 & 12 & 28 & 17 \\ 1 & 15 & 23 & 26 \\ 5 & 18 & 31 & 10 \\ 2 & 8 & 24 & 14 \\ 32 & 27 & 3 & 9 \\ 19 & 13 & 30 & 6 \\ 22 & 11 & 4 & 25 \end{bmatrix}$$

S 盒中每一个元素都是 4 bit 的比特串，输入以 1, 6, 2, 3, 4, 5 的顺序重排序，然后输出表中对应位置的值。

$$S1 = \begin{bmatrix} 14 & 4 & 13 & 1 & 2 & 15 & 11 & 8 & 3 & 10 & 6 & 12 & 5 & 9 & 0 & 7 \\ 0 & 15 & 7 & 4 & 14 & 2 & 13 & 1 & 10 & 6 & 12 & 11 & 9 & 5 & 3 & 8 \\ 4 & 1 & 14 & 8 & 13 & 6 & 2 & 11 & 15 & 12 & 9 & 7 & 3 & 10 & 5 & 0 \\ 15 & 12 & 8 & 2 & 4 & 9 & 1 & 7 & 5 & 11 & 3 & 14 & 10 & 0 & 6 & 13 \end{bmatrix}$$

$$S2 = \begin{bmatrix} 15 & 1 & 8 & 14 & 6 & 11 & 3 & 4 & 9 & 7 & 2 & 13 & 12 & 0 & 5 & 10 \\ 3 & 13 & 4 & 7 & 15 & 2 & 8 & 14 & 12 & 0 & 1 & 10 & 6 & 9 & 11 & 5 \\ 0 & 14 & 7 & 11 & 10 & 4 & 13 & 1 & 5 & 8 & 12 & 6 & 9 & 3 & 2 & 15 \\ 13 & 8 & 10 & 1 & 3 & 15 & 4 & 2 & 11 & 6 & 7 & 12 & 0 & 5 & 14 & 9 \end{bmatrix}$$

$$S3 = \begin{bmatrix} 10 & 0 & 9 & 14 & 6 & 3 & 15 & 5 & 1 & 13 & 12 & 7 & 11 & 4 & 2 & 8 \\ 13 & 7 & 0 & 9 & 3 & 4 & 6 & 10 & 2 & 8 & 5 & 14 & 12 & 11 & 15 & 1 \\ 13 & 6 & 4 & 9 & 8 & 15 & 3 & 0 & 11 & 1 & 2 & 12 & 5 & 10 & 14 & 7 \\ 1 & 10 & 13 & 0 & 6 & 9 & 8 & 7 & 4 & 15 & 14 & 3 & 11 & 5 & 2 & 12 \end{bmatrix}$$

$$S4 = \begin{bmatrix} 7 & 13 & 14 & 3 & 0 & 6 & 9 & 10 & 1 & 2 & 8 & 5 & 11 & 12 & 4 & 15 \\ 13 & 8 & 11 & 5 & 6 & 15 & 0 & 3 & 4 & 7 & 2 & 12 & 1 & 10 & 14 & 9 \\ 10 & 6 & 9 & 0 & 12 & 11 & 7 & 13 & 15 & 1 & 3 & 14 & 5 & 2 & 8 & 4 \\ 3 & 15 & 0 & 6 & 10 & 1 & 13 & 8 & 9 & 4 & 5 & 11 & 12 & 7 & 2 & 14 \end{bmatrix}$$

$$S5 = \begin{bmatrix} 2 & 12 & 4 & 1 & 7 & 10 & 11 & 6 & 8 & 5 & 3 & 15 & 13 & 0 & 14 & 9 \\ 14 & 11 & 2 & 12 & 4 & 7 & 13 & 1 & 5 & 0 & 15 & 10 & 3 & 9 & 8 & 6 \\ 4 & 2 & 1 & 11 & 10 & 13 & 7 & 8 & 15 & 9 & 12 & 5 & 6 & 3 & 0 & 14 \\ 11 & 8 & 12 & 7 & 1 & 14 & 2 & 13 & 6 & 15 & 0 & 9 & 10 & 4 & 5 & 3 \end{bmatrix}$$

$$S6 = \begin{bmatrix} 12 & 1 & 10 & 15 & 9 & 2 & 6 & 8 & 0 & 13 & 3 & 4 & 14 & 7 & 5 & 11 \\ 10 & 15 & 4 & 2 & 7 & 12 & 9 & 5 & 6 & 1 & 13 & 14 & 0 & 11 & 3 & 8 \\ 9 & 14 & 15 & 5 & 2 & 8 & 12 & 3 & 7 & 0 & 4 & 10 & 1 & 13 & 11 & 6 \\ 4 & 3 & 2 & 12 & 9 & 5 & 15 & 10 & 11 & 14 & 1 & 7 & 6 & 0 & 8 & 13 \end{bmatrix}$$

$$S7 = \begin{bmatrix} 4 & 11 & 2 & 14 & 15 & 0 & 8 & 13 & 3 & 12 & 9 & 7 & 5 & 10 & 6 & 1 \\ 13 & 0 & 11 & 7 & 4 & 9 & 1 & 10 & 14 & 3 & 5 & 12 & 2 & 15 & 8 & 6 \\ 1 & 4 & 11 & 13 & 12 & 3 & 7 & 14 & 10 & 15 & 6 & 8 & 0 & 5 & 9 & 2 \\ 6 & 11 & 13 & 8 & 1 & 4 & 10 & 7 & 9 & 5 & 0 & 15 & 14 & 2 & 3 & 12 \end{bmatrix}$$

$$S8 = \begin{bmatrix} 13 & 2 & 8 & 4 & 6 & 15 & 11 & 1 & 10 & 9 & 3 & 14 & 5 & 0 & 12 & 7 \\ 1 & 15 & 13 & 8 & 10 & 3 & 7 & 4 & 12 & 5 & 6 & 11 & 0 & 14 & 9 & 2 \\ 7 & 11 & 4 & 1 & 9 & 12 & 14 & 2 & 0 & 6 & 10 & 13 & 15 & 3 & 5 & 8 \\ 2 & 1 & 14 & 7 & 4 & 10 & 8 & 13 & 15 & 12 & 9 & 0 & 3 & 5 & 6 & 11 \end{bmatrix}$$

3.1.4 密钥扩展

密钥扩展 (Key Schedule) 是 DES 加密的一个重要步骤, 它将 64 位的密钥扩展为 16 个 48 位的子密钥。密钥扩展的过程包括:

1. PC-1 (Permuted Choice 1), 将 64 位密钥转换为 56 位。
2. 分为两部分 (C 和 D), 每部分 28 位, 进行 16 轮迭代。
3. 两部分分别循环左移 (Left Shift), 当 $i=1, 2, 9, 16$ 时, 左移一次, 其他轮次左移两次。
4. 使用 PC-2 生成 48 位子密钥。

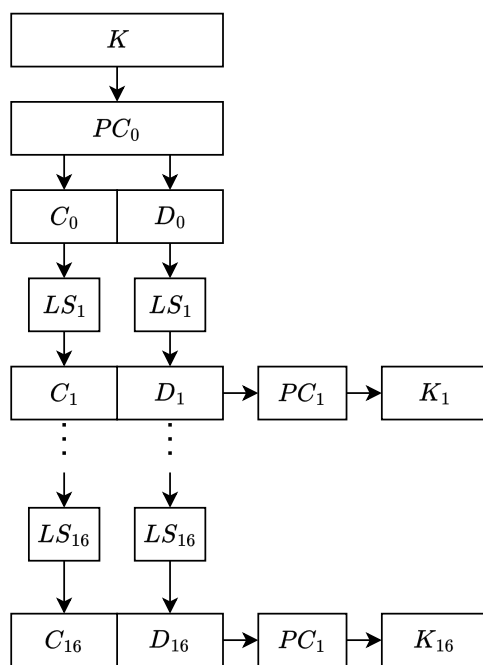


图 3.3: 密钥扩展

$$PC_1 = \begin{bmatrix} 57 & 49 & 41 & 33 & 25 & 17 & 9 \\ 1 & 58 & 50 & 42 & 34 & 26 & 18 \\ 10 & 2 & 59 & 51 & 43 & 35 & 27 \\ 19 & 11 & 3 & 60 & 52 & 44 & 36 \\ 63 & 55 & 47 & 39 & 31 & 23 & 15 \\ 7 & 62 & 54 & 46 & 38 & 30 & 22 \\ 14 & 6 & 61 & 53 & 45 & 37 & 29 \\ 21 & 13 & 5 & 28 & 20 & 12 & 4 \end{bmatrix} \quad PC_2 = \begin{bmatrix} 14 & 17 & 11 & 24 & 1 & 5 \\ 3 & 28 & 15 & 6 & 21 & 10 \\ 23 & 19 & 12 & 4 & 26 & 8 \\ 16 & 7 & 27 & 20 & 13 & 2 \\ 41 & 52 & 31 & 37 & 47 & 55 \\ 30 & 40 & 51 & 45 & 33 & 48 \\ 44 & 49 & 39 & 56 & 34 & 53 \\ 46 & 42 & 50 & 36 & 29 & 32 \end{bmatrix}$$

3.1.5 TDES

DES 极大缺陷是它使用了短密钥，为了克服这一点，Tuchman 在 1979 年提出了三重 DES (Triple DES, TDES)，它通过对数据进行三次 DES 加密来提高安全性，密钥长度为 168 位，当 $k_1 = k_2 = k_3$ ，TDES 退化成 DES， $k = (k_1, k_2, k_3)$ ，加密过程为：

$$TDES(m) = DES_{k_3}(DES_{k_2}^{-1}(DES_{k_1}(m)))$$

3.2 AES

3.2.1 AES 简介

1997 年 1 月，NIST 开始在全世界征集 DES 替代算法，2001 年 10 月，NIST 宣布 Rijndael 算法为 AES 标准 (Advanced Encryption Standard)。AES 支持 128 位、192 位和 256 位的密钥长度，分组长度为 128 位，加密多轮 (10 轮、12 轮或 14 轮，取决于密钥长度)。AES 中所有操作都以字节 (8 位) 为单位，即所有运算在 $GF(2^8)$ 上进行。

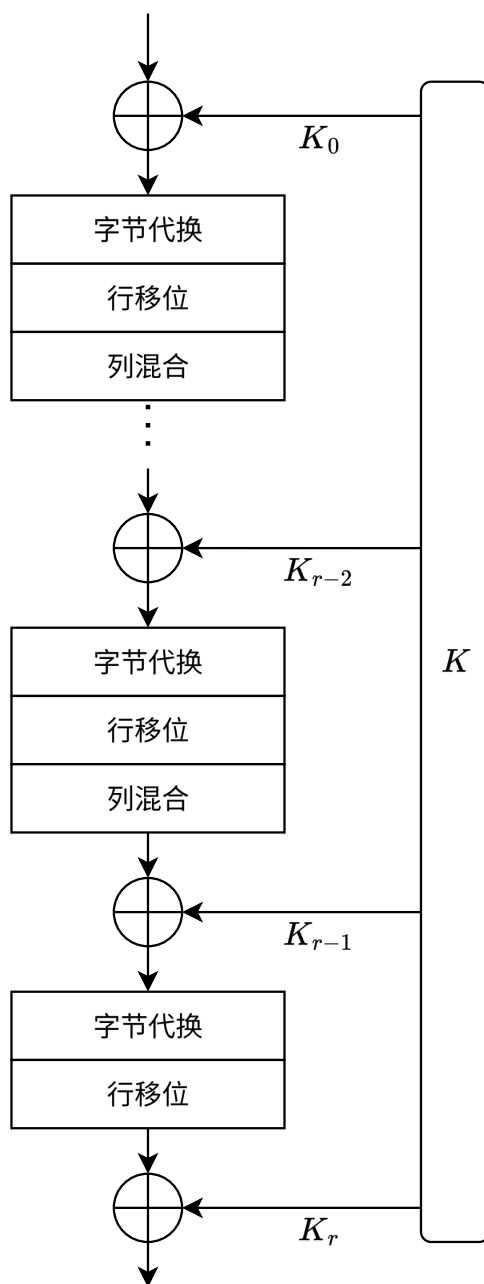


图 3.4: AES 加密流程

3.2.2 轮函数

AES 的轮函数包括四个主要步骤（最后一轮没有列混合）：

1. 字节代换 (SubBytes)：使用 S 盒对每个字节进行非线性代换。
2. 行移位 (ShiftRows)：输入分为四行，形成状态矩阵，每行进行循环左移 (0, 1, 2, 3 字节)。

3. 列混合 (MixColumns): 对状态矩阵每列进行线性变换。

$$M = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix}$$

4. 轮密钥加 (AddRoundKey): 将当前轮的密钥与输入进行异或操作。

$$S = \begin{bmatrix} 63 & 7C & 77 & 7B & F2 & 6B & 6F & C5 & 30 & 01 & 67 & 2B & FE & D7 & AB & 76 \\ CA & 82 & C9 & 7D & FA & 59 & 47 & F0 & AD & D4 & A2 & AF & 9C & A4 & 72 & C0 \\ B7 & FD & 93 & 26 & 36 & 3F & F7 & CC & 34 & A5 & E5 & F1 & 71 & D8 & 31 & 15 \\ 04 & C7 & 23 & C3 & 18 & 96 & 05 & 9A & 07 & 12 & 80 & E2 & EB & 27 & B2 & 75 \\ 09 & 83 & 2C & 1A & 1B & 6E & 5A & A0 & 52 & 3B & D6 & B3 & 29 & E3 & 2F & 84 \\ 53 & D1 & 00 & ED & 20 & FC & B1 & 5B & 6A & CB & BE & 39 & 4A & 4C & 58 & CF \\ D0 & EF & AA & FB & 43 & 4D & 33 & 85 & 45 & F9 & 02 & 7F & 50 & 3C & 9F & A8 \\ 51 & A3 & 40 & 8F & 92 & 9D & 38 & F5 & BC & B6 & DA & 21 & 10 & FF & F3 & D2 \\ CD & 0C & 13 & EC & 5F & 97 & 44 & 17 & C4 & A7 & 7E & 3D & 64 & 5D & 19 & 73 \\ 60 & 81 & 4F & DC & 22 & 2A & 90 & 88 & 46 & EE & B8 & 14 & DE & 5E & 0B & DB \\ E0 & 32 & 3A & 0A & 49 & 06 & 24 & 5C & C2 & D3 & AC & 62 & 91 & 95 & E4 & 79 \\ E7 & C8 & 37 & 6D & 8D & D5 & 4E & A9 & 6C & 56 & F4 & EA & 65 & 7A & AE & 08 \\ BA & 78 & 25 & 2E & 1C & A6 & B4 & C6 & E8 & DD & 74 & 1F & 4B & BD & 8B & 8A \\ 70 & 3E & B5 & 66 & 48 & 03 & F6 & 0E & 61 & 35 & 57 & B9 & 86 & C1 & 1D & 9E \\ E1 & F8 & 98 & 11 & 69 & D9 & 8E & 94 & 9B & 1E & 87 & E9 & CE & 55 & 28 & DF \\ 8C & A1 & 89 & 0D & BF & E6 & 42 & 68 & 41 & 99 & 2D & 0F & B0 & 54 & BB & 16 \end{bmatrix}$$

S 盒其实是一个代数变换, 可以写成两个阶段的 $GF(2^8)$ 上的运算组合:

1. 计算输入字节的乘法逆元 (0 映射到 0)。
2. 对逆元进行仿射变换 (从低位到高位排), $y = A \cdot x \oplus b$ 。

$$A = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}, \quad b = \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}$$

3.2.3 密钥扩展

AES 的密钥扩展 (Key Expansion) 将原始密钥扩展为多个轮密钥。对于 128 位密钥, 生成 10 个轮密钥; 192 位密钥, 生成 12 个轮密钥; 256 位密钥, 生成 14 个轮密钥。主要步骤包括: 先将原始密钥分为 N_k 个 W (Word, 32 bits), 之后迭代生成新的 W, 直到数量满足需要。当 $N_k = 4, 6$,

$$W_i = \begin{cases} W_{i-N_k} \oplus \text{SubByte}(\text{RotByte}(W_{i-1})) \oplus \text{Rcon}_{i/N_k}, & i \bmod N_k = 0 \\ W_{i-N_k} \oplus W_{i-1}, & i \bmod N_k \neq 0 \end{cases}$$

其中, RotByte 是将 Word 循环左移 1 字节, SubByte 是 S 盒替换, Rcon 是轮常量 (Round Constant), $\text{Rcon}_i = \begin{bmatrix} 2^{i-1} & 00 & 00 & 00 \end{bmatrix}$ 。当 $N_k = 8$, 不一样的一点是当 $i \equiv 4 \pmod{N_k}$, $W_i = W_{i-N_k} \oplus \text{SubByte}(W_{i-1})$ 。

3.3 IDEA

3.3.1 IDEA 简介

IDEA (International Data Encryption Algorithm) 是 1991 年由 James Massey 和 Xuejia Lai 提出的分组密码算法, 原名为 IPES (International Pseudo-Efficient Scheme)。IDEA 使用 128 位密钥, 分组长度为 64 位, 采用 8 轮加密。

3.3.2 轮函数

IDEA 轮函数不是 Feistel 结构, 其中主要有三种运算: 异或、模 2^{16} 加法和模 $2^{16} + 1$ 乘法。每轮将输入分为 4 个 16 位分组, 与 6 个 16 位子密钥进行运算。

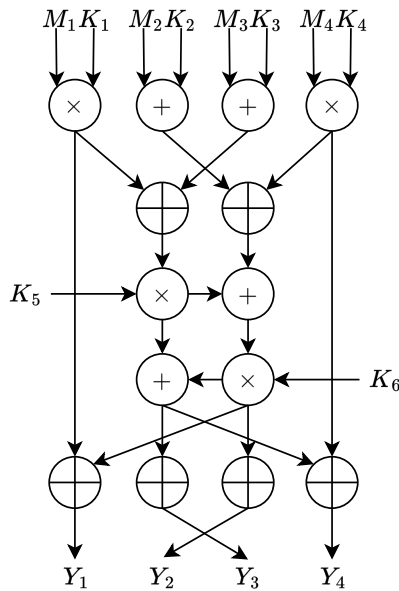


图 3.5: IDEA 轮结构

3.3.3 输出变换

8 轮加密后, IDEA 使用 4 个子密钥进行输出变换, 注意 M_2 与 M_3 交换了位置。

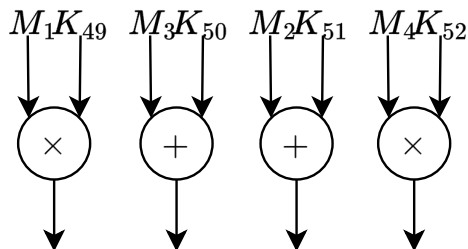


图 3.6: 输出变换

3.3.4 子密钥产生

IDEA 的子密钥由 128 位密钥生成, 分为 52 个 16 位子密钥。子密钥生成过程如下:

1. 将 128 位密钥分为 8 个 16 位分组。
2. 对每个分组进行循环左移 25 位, 生成子密钥。
3. 生成的子密钥按顺序使用。

3.4 SM4

3.4.1 SM4 简介

SM4 是中国国家密码管理局于 2012 年发布的分组密码算法, 分组长度为 128 位, 密钥长度为 128 位, 采用 32 轮加密。SM4 的加解密步骤是一样的, 只不过轮密钥顺序相反。

3.4.2 轮函数

先将明文分为 4 个 32 位分组, 然后进行 32 轮加密, 每轮使用一个 32 位的子密钥, 生成一个新的 32 位分组。

$$X_{i+4} = X_i \oplus T(X_{i+1} \oplus X_{i+2} \oplus X_{i+3} \oplus RK_i)$$

合成置换 T 是一个可逆置换, 由非线性变换 S 和线性变换 L 组成, S 使用 S 盒进行以字节为单位的置换。

$$S \begin{bmatrix} D6 & 90 & E9 & FE & CC & E1 & 3D & B7 & 16 & B6 & 14 & C2 & 28 & FB & 2C & 05 \\ 2B & 67 & 9A & 76 & 2A & BE & 04 & C3 & AA & 44 & 13 & 26 & 49 & 86 & 06 & 99 \\ 9C & 42 & 50 & F4 & 91 & EF & 98 & 7A & 33 & 54 & 0B & 43 & ED & CF & AC & 62 \\ E4 & B3 & 1C & A9 & C9 & 08 & E8 & 95 & 80 & DF & 94 & FA & 75 & 8F & 3F & A6 \\ 47 & 07 & A7 & FC & F3 & 73 & 17 & BA & 83 & 59 & 3C & 19 & E6 & 85 & 4F & A8 \\ 68 & 6B & 81 & B2 & 71 & 64 & DA & 8B & F8 & EB & 0F & 4B & 70 & 56 & 9D & 35 \\ 1E & 24 & 0E & 5E & 63 & 58 & D1 & A2 & 25 & 22 & 7C & 3B & 01 & 21 & 78 & 87 \\ D4 & 00 & 46 & 57 & 9F & D3 & 27 & 52 & 4C & 36 & 02 & E7 & A0 & C4 & C8 & 9E \\ EA & BF & 8A & D2 & 40 & C7 & 38 & B5 & A3 & F7 & F2 & CE & F9 & 61 & 15 & A1 \\ E0 & AE & 5D & A4 & 9B & 34 & 1A & 55 & AD & 93 & 32 & 30 & F5 & 8C & B1 & E3 \\ 1D & F6 & E2 & 2E & 82 & 66 & CA & 60 & C0 & 29 & 23 & AB & 0D & 53 & 4E & 6F \\ D5 & DB & 37 & 45 & DE & FD & 8E & 2F & 03 & FF & 6A & 72 & 6D & 6C & 5B & 51 \\ 8D & 1B & AF & 92 & BB & DD & BC & 7F & 11 & D9 & 5C & 41 & 1F & 10 & 5A & D8 \\ 0A & C1 & 31 & 88 & A5 & CD & 7B & BD & 2D & 74 & D0 & 12 & B8 & E5 & B4 & B0 \\ 89 & 69 & 97 & 4A & 0C & 96 & 77 & 7E & 65 & B9 & F1 & 09 & C5 & 6E & C6 & 84 \\ 18 & F0 & 7D & EC & 3A & DC & 4D & 20 & 79 & EE & 5F & 3E & D7 & CB & 39 & 48 \end{bmatrix}$$

$$L(B) = B \oplus (B \lll 2) \oplus (B \lll 10) \oplus (B \lll 18) \oplus (B \lll 24)$$

最终密文是倒序的 4 个 32 个分组拼接而成。

3.4.3 密钥扩展

SM4 的密钥扩展先把密钥分为 4 个 32 位子密钥 MK，再使用其与 FK 生成中间变量 K， $K_i = MK_i \oplus FK_i$ 。

$$FK = \begin{bmatrix} A3B1BAC6 & 56AA3350 & 677D9197 & B27022DC \end{bmatrix}$$

之后的步骤类似轮函数的操作，最后生成的轮密钥 $RK_i = K_{i+4}$ 。

$$K_{i+4} = K_i \oplus T'(K_{i+1} \oplus K_{i+2} \oplus K_{i+3} \oplus CK_i)$$

$$CK = \begin{bmatrix} 00070E15 & 1C232A31 & 383F464D & 545B6269 \\ 70777E85 & 8C939AA1 & A8AFB6BD & C4CBD2D9 \\ E0E7EEF5 & FC030A11 & 181F262D & 343B4249 \\ 50575E65 & 6C737A81 & 888F969D & A4ABB2B9 \\ C0C7CED5 & DCE3EAF1 & F8FF060D & 141B2229 \\ 30373E45 & 4C535A61 & 686F767D & 848B9299 \\ A0A7AEB5 & BCC3CAD1 & D8DFE6ED & F4FB0209 \\ 10171E25 & 2C333A41 & 484F565D & 646B7279 \end{bmatrix}$$

$$T'(A) = L'(S(A)), \quad L'(B) = B \oplus (B \lll 13) \oplus (B \lll 23)$$

4 哈希函数

4.1 哈希函数简介

密码学里的哈希函数用来保证数据完整性，它将任意长度的输入数据转换为固定长度的输出值，这个输出被称为输入消息的消息摘要（Message Digest, MD）。Hash 函数有带密钥的和不带密钥的，带密钥的哈希函数通常用作消息认证码（Message Authentication Code, MAC）。

在密码学中，一般要求 Hash 函数满足以下三个性质：

- 单向性，也称抗原像性（Pre-image Resistance）：给定一个哈希值，反推原始输入在计算上是困难的。
- 弱抗碰撞（Weakly Collision Resistance）：已知 x ，寻找 $x' \neq x$ ，使得 $h(x') = h(x)$ 在计算上是困难的。
- 强抗碰撞（Strongly Collision Resistance）：寻找 x 和 x' ($x' \neq x$)，使得 $h(x') = h(x)$ 在计算上是困难的。

4.2 MD5

MD5 是 1991 年由密码学家 Ronald Rivest 设计的 Hash 算法，其分组长度是 512 位，输出 128 位的消息摘要。

1. 消息填充，填充 1 后跟若干 0，直至原消息可以分为若干 512 位的块和最后一个 448 位的块。
2. 取原消息的长度（以 bit 为单位）的最低 64 位添加到填充的消息之后。
3. 每个 512 位的块分成 16 个单字 M ，使用 4 个 32 位寄存器（A, B, C, D）做运算，初始值：

$$A = 67452301 \quad B = efcdab89 \quad C = 98badcfe \quad D = 10325476$$

4. 做 4 轮运算，每轮 16 次操作。

$$A = B + ((A + f(B, C, D) + M_k + T_i) \lll s) \quad (A, B, C, D) \leftarrow (D, A, B, C)$$

注意每次操作后有个循环右移寄存器值的操作， f 在这 4 轮中分别代表以下运算：

$$F(B, C, D) = (B \wedge C) \vee (\neg B \wedge D) \quad G(B, C, D) = (B \wedge D) \vee (C \wedge \neg D)$$

$$H(B, C, D) = B \oplus C \oplus D \quad I(B, C, D) = C \oplus (B \vee \neg D)$$

T_i 定义 (i 从 0 到 63): $T_i = \lfloor 2^{32} \cdot |\sin(i+1)| \rfloor$, s 和 k 每轮值不一样, s 在各轮中循环使用：

$$s = \{7, 12, 17, 22\}, \{5, 9, 14, 20\}, \{4, 11, 16, 23\}, \{6, 10, 15, 21\}$$

$$k = \{i\}, \{(5i+1) \bmod 16\}, \{(3i+5) \bmod 16\}, \{(7i) \bmod 16\}$$

5. 每个块运算过后，都将此次得到的寄存器值与原值相加存入寄存器，最后拼接寄存器的值作为 MD5 的输出。

4.3 SHA-1

SHA (Secure Hash Algorithm) 是美国国家安全局 (National Security Agency, NSA) 于 1993 年在 MD4 基础上改进设计的, 并由 NIST 公布作为安全哈希标准 (Secure Hash Standard)。1995 年, NSA 提出 SHA 的改进算法 SHA-1, SHA 和 SHA-1 输出长度都为 160 位, 2002 年, SHA 的变形算法 SHA-256、SHA-384 和 SHA-512 公布。

SHA-1 是从 MD4 改进而来, 算法与 MD5 算法有许多相似之处:

1. SHA-1 的消息填充与分块与 MD5 一致。
2. 通过扩展, 将 16 个 M 块扩展成 80 个, $M_i = (M_{i-3} \oplus M_{i-8} \oplus M_{i-14} \oplus W_{i-16}) \lll 1$ 。
3. 使用 5 个 32 位寄存器 (A, B, C, D, E) 做运算, 初始值:

$$A = 67452301 \quad B = efcdab89 \quad C = 98badcfe \quad D = 10325476 \quad E = c3d2e1f0$$

4. 做 4 轮运算, 每轮 20 次操作。

$$\begin{aligned} TEMP &= (A \lll 5) + f(B, C, D) + E + M_i + K \\ E &= D \quad D = C \quad C = (B \lll 30) \quad B = A \quad A = TEMP \end{aligned}$$

K 循环四个值: $\{5a827999, 6ed9eba1, 8f1bbcdc, ca62c1d6\}$, f 在这 4 轮中分别代表以下运算:

$$\begin{aligned} f_1(B, C, D) &= (B \wedge C) \vee (\neg B \wedge D) & f_2(B, C, D) &= B \oplus C \oplus D \\ f_3(B, C, D) &= (B \wedge C) \vee (B \wedge D) \vee (C \wedge D) & f_4(B, C, D) &= B \oplus C \oplus D \end{aligned}$$

5. 和 MD5 一样, 也将每次得到的值与原值相加存入寄存器, 最后拼接寄存器的值作为输出。

4.4 MD6

MD5 被发现存在安全漏洞后, 在 Crypto2008 上, Rivest 提出了 MD6 算法, 该算法每轮输出 1024 位, 支持四种摘要长度, 即 MD6-224、MD6-256、MD6-384 和 MD6-512。

4.5 SHA-2、SHA-3 及 SM3 简介

SHA-2 指在 SHA-1 基础上形成的 SHA-224、SHA-256、SHA-384、SHA-512 等一系列算法, SHA-3 是 2009 年 NIST 公开征集安全性更高的算法。SM3 是王小云等设计的, 于 2010 年被中国国家密码管理局颁布为标准, SM3 与 SHA-256 的算法有相似之处, 它的安全性和效率与其相当。